

公司 TODO 管理体系

基于 haitun agent 的整体架构方案

飞书平台 · 目标与 todo 双周期管理 · LLM wiki 工作树 · mentor 评价闭环

代码基线: psi-agent origin/main (fd6ef94f) | 文档日期: 2026-08-05

0. 方案摘要

本方案把「每周一三五更新 todo」这件事做成一条可自动运转的闭环：从飞书文档表格采集事实，汇成一棵以 boss 为根的 LLM wiki 工作树，向 mentor 推结构化报表、向执行人派可勾选的 todo 卡与飞书任务，回收 mentor 评价并写回 wiki，最后做闭环判定、把未闭环项带着逾期天数回流下一周期。

关键取舍：改动全部落在 agent 包内——三个 SKILL、两个新工具、三条 schedule，框架代码一行不改。新工具只加在「工具本身就是约束」的地方（请假日期区间判定、per-mentor 台账的开通与授权），其余一律复用 main 上已有的工具。三处能力边界写在第 8 节，均已定案。

阶段	落在哪	主要工具	触发时机
采集	飞书文档表格（人工事实源）	feishu_doc_read / feishu_sheet_read	周一三五 15:00
判假	同表的「请假表」子表	feishu_leave_query（新增）	同上，采集后立即
建树	LLM wiki 工作树	wiki_write / wiki_read / wiki_links	同上
报表	per-mentor 独立 Bitable 台账 + 统计卡	feishu_mentor_ledger_ensure（新增） / feishu_bitable_* / feishu_chart	同上
派发	TODO 卡 + 飞书原生任务	feishu_todo_card_send / feishu_task_create	同上
交付	执行人勾选提交	feishu_todo_card_tick	各 todo 自设截止前
评价	mentor 打分评语	feishu_message_send_card + wiki_write	交付后 / 周期末
闭环	五要素判定与回流	wiki_read + feishu_task_get	下一周期开始前

1. 需求归纳与拆解

把你的两个场景拆成七件可独立验收的事，后续每一节对应其中一件或多件：

- R1 采集：每周一三五固定时间，读取那张飞书文档表格里每个人的整体 todo list（列=日期，行=人，含负责人与 mentor 标注）。

- R2 建树：按 boss → 项目负责人 → 项目内基础工作人员，建成一棵 LLM wiki 工作树，每个节点是一个人。
- R3 累加：节点内容按时间排序、定时更新，且只增不覆盖历史；同时允许随时人工修改目标。
- R4 报表：同一时间把每个人的 todo 汇成看板/报表发给其 mentor；mentor 只收自己手下所有人的，boss 不写 todo。
- R5 派发：把每条 todo 作为任务要求发给该 todo 的负责人（即写这条 todo 的人）；请假的人必须被明确标出。
- R6 交付与评价：负责人按自设时间点交付；mentor 对完成情况给出评价，评价由 agent 写回该人 wiki 对应 todo 之后。
- R7 闭环与打分：以闭环为核心，只有走完整圈才算交付完成；agent 依据闭环情况辅助打分，但以 mentor 打分为准。

两个必须先说清的语义：大目标要求「与友商的对比」和「具体外部成果（用户数/金额）」，这两项是大目标的必填字段，缺失时本方案的做法是照常入库但在报表里标为「待补」，不阻塞整条流水线；boss 作为根节点只读汇总、不参与写 todo 与被派发。

2. 总体架构

四层分工：调度层只管「什么时候动」，编排层只管「按什么顺序调工具」，能力层以 main 上已有的工具为主、另加两个新工具，存储层是四个事实源。约束是「不改框架代码」——所有新增都落在 agent 包的 tools/ 与 skills/ 目录内，因此升级框架不会冲突。

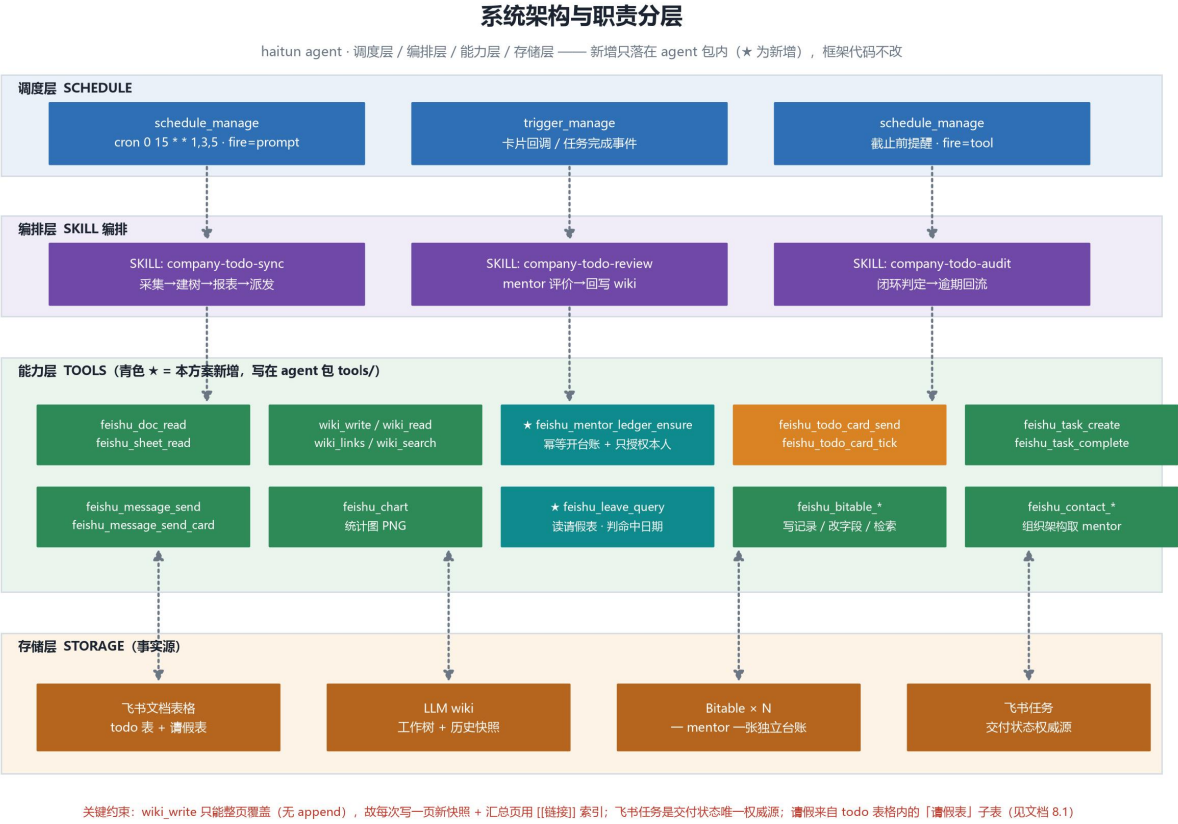


图 1 系统架构与职责分层

2.1 四个存储各自的权威边界

同一份信息只认一个权威源，避免两处不一致时无法裁决：

存储	权威内容	谁写	冲突时以谁为准
飞书文档表格	每人填报的原始目标与 todo 文本	员工人工填写	原始文本以它为准
LLM wiki	工作树结构、历史快照、mentor 评价	agent 写；人可随时改	目标口径以它为准
Bitable 台账	结构化 todo 行、状态、打分	agent 写	统计与筛选以它为准
飞书任务	某条 todo 到底交付没有	执行人勾选	交付状态以它为准

2.2 新增的两个工具 (agent 包内，不改框架)

本仓 feishu_api 的模块注释写明了取舍原则：形状容易写错、需要被强制保证的调用才做成专用工具，其余交给通用工具加技能表。下面两件事都属于前者——不是为了省字，而是为了让判定唯一。

工具	签名要点	为什么必须是工具而不是技能里的一段话
feishu_leave_query	(sheet_token, sheet_name,	日期区间重叠判定是纯逻辑，写在技能里等于让模型

	date_from, date_to, names_json="") → 每人的请假区间与命中日期	每周期心算一次日历，错一次就给休假的人派活并记逾期。工具把「哪天算请假」固定下来。
feishu_mentor_ledger_ensure	(mentor_open_id, mentor_name, folder_token, template_app_token="") → 该 mentor 的 app_token / table_id	开通要幂等（重复跑不能建出第二张台账）、授权要恰好一次、table_id 必须解析而不是猜。三个动作缺一个就出错，且错误不会立刻暴露。

两个工具都只组合已有端点：前者走 feishu_sheet_read，后者走 POST /open-apis/bitable/v1/apps/:app_token/copy 与 POST /open-apis/drive/v1/permissions/:token/members（端点写法见 feishu-bitable 与 feishu-permission 两个技能）。

2.3 新增的三个技能

技能	触发	职责边界
company-todo-sync	周一三五 15:00	采集表格 → 判请假 → 建/更 wiki 快照 → 写各 mentor 台账 → 推报表 → 派卡与任务
company-todo-review	交付事件驱动	收 mentor 评价卡 → 写台账打分字段 → 追加到 wiki 快照页对应 todo 之后
company-todo-audit	次日 10:00	闭环五要素判定 → 未闭环项带逾期天数回流下一周期 → 给 boss 出整体统计

3. LLM wiki 工作树设计 (R2 / R3)

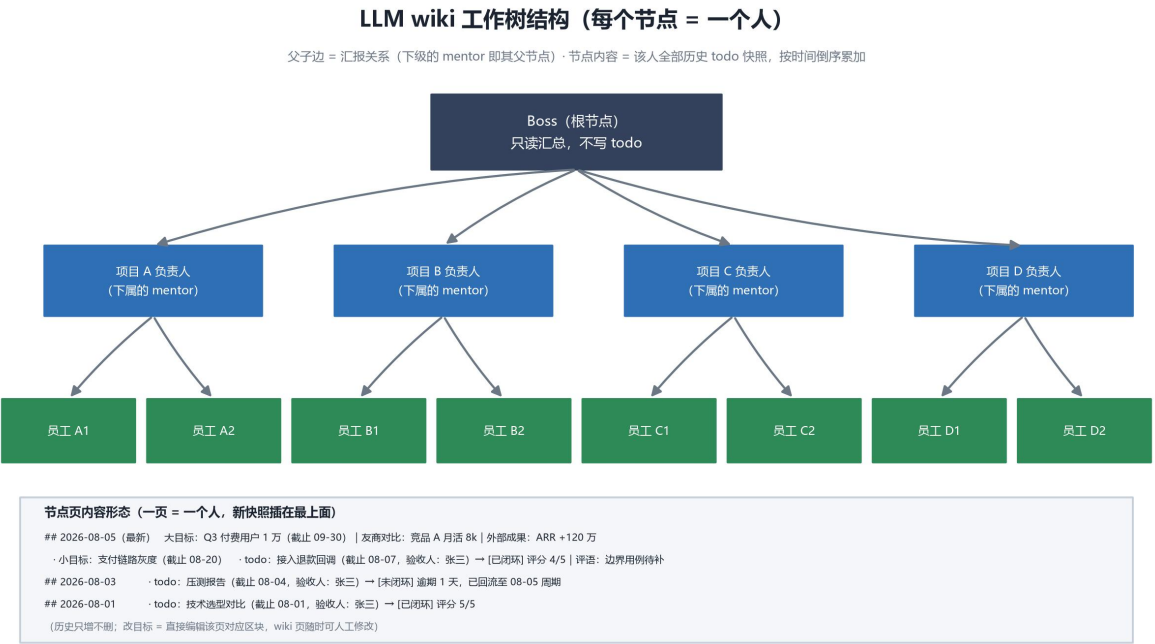


图 2 wiki 工作树结构与节点内容形态

3.1 页面命名与树结构

树不靠目录层级表达，而靠 wiki 的 [[双向链接]] 表达——因为 wiki_links 能直接 traverse，而目录嵌套在 wiki 工具里没有对应查询。每人一页，页内用「上级」「下属」两个区块声明汇报关系：

- 人员页：《张三》——正文首段写「上级：[[李四]]」「下属：[[王五]]、[[赵六]]」，其后是按时间倒序的历史快照。
- 组织根页：《公司工作树》——只放 boss 一个 [[链接]] 与一句读法说明，作为遍历入口。
- 周期索引页：《TODO 周期 2026-08-05》——列出本次采集覆盖的所有人 [[链接]] 与本次统计数，便于回溯某一次运行。

tags 统一打三类：person / cycle / project，配合 wiki_list 与 wiki_search 做筛选。

3.2 只增不覆盖怎么做到（这是本节最关键的一条）

wiki_write 只有整页覆盖、没有 append 模式（已在 main 上核实）。所以「只累加」必须由编排层自己保证，方案采用你选定的「每次一页 + 汇总页索引」：

- 每个周期为每人写一页独立快照页：《张三 todo 2026-08-05》，内容是该次填报的完整目标与 todo。这一页此后不再被任何自动流程改写。
- 人员汇总页《张三》只被改写「历史索引」区块：在最上面插入一条 [[张三 todo 2026-08-05]]，历史条目原样保留。
- 改写汇总页前先 wiki_read 取回全文，只在索引区块顶部插入新行，其余字节原样写回——即「读回全文再整页回写」。
- 同一人同一周期重复运行是幂等的：快照页 slug 相同，重写即覆盖同一次的内容；汇总页发现索引已存在该条则不重复插入。

并发注意：同一个人的汇总页不要在两个并发任务里同时改写（后写的会覆盖先写的）。编排层按人串行处理即可规避，规模在数百人量级下耗时可接受。

3.3 人工修改目标如何不被自动流程冲掉

把「可人工编辑」和「自动写入」放在不同页上，是这套设计能同时满足 R3 两个要求的原因：

- 人要改目标口径：直接改人员汇总页《张三》的「当前目标」区块。自动流程只碰它的「历史索引」区块，两者不重叠。
- 历史快照页原则上不改；确实要改（填错了）就直接编辑那一页，下次运行不会覆盖它。

- agent 每次写入前 wiki_read 一次，把人工修改后的「当前目标」读进上下文，作为本次报表口径——所以人改完立刻生效。

4. 采集层设计 (R1)

已有 SKILL 「feishu-todo-board-sync」做的正是「读个人 ToDoList 文档 → 拆到人 → 写进团队看板表格」，本方案复用它的三条硬规则，不重写解析逻辑：结构每次现场探（表头行/人名列/SHEET_ID 都读出来，不写死）、归属按行内 @人名 拆、目标单元格非空先报警不覆盖。

4.1 采集流程

- 1. wiki 链接换 token: feishu_wiki_get_node 把 wiki 链接换成 obj_token（表格通常挂在 wiki 下）。
- 2. 探结构: feishu_sheet_tabs 查 SHEET_ID; feishu_sheet_read 读表头行认「日期列」、读首列认「人名行」，同时读出每人的 mentor 标注列。
- 3. 定位本周期列: 按今天日期在表头里找对应列；找不到就取最接近的、并在报表里注明「按 08-05 列读取」。
- 4. 逐人取值: 读该列每一行的单元格，得到每人本次填报的整段文本（大目标 / 小目标 / todo 三级）。
- 5. 解析分层: 按缩进与关键词把整段文本切成大目标、小目标、todo 三级，各自抽出截止日期与验收人。
- 6. 校验并落库: 缺验收人、缺截止日、大目标缺友商对比或外部成果的，标 needs_fix 但照常入库。

4.2 解析字段规范

层级	必填	选填	缺失时处理
大目标	标题、截止日期	友商对比、外部成果（用户/金额）	入库并标「待补」，报表高亮
小目标	标题、截止日期	所属大目标	无归属则挂到该人当期唯一大目标
todo	标题、截止日期、验收人	所属小目标、成果物形式	缺验收人则取该人 mentor 兜底

验收人兜底规则值得单独说明：todo 缺验收人时取该人 mentor，因为你的规则里「一般是自己的上级 mentor」，兜底后在报表标注「验收人由 mentor 兜底」，让 mentor 有机会纠正。

5. mentor 报表设计 (R4)

载体用多维表格 (Bitable) 做台账 + 一张统计卡片做摘要。选 Bitable 而非纯电子表格，是因为状态与打分需要结构化字段来筛选和统计，而 mentor 只该看到自己下属——这靠筛选视图实现。

5.1 台账表结构

字段	类型	说明
周期日期	日期	本次采集的周期，如 2026-08-05
负责人	人员	写这条 todo 的人，也是被派发对象
mentor	人员	验收人，报表筛选的关键字段
层级	单选	大目标 / 小目标 / todo
父项	文本	所属上级目标标题，用于还原三级结构
标题	文本	目标或 todo 正文
截止日期	日期	自设交付时间点
状态	单选	待开始 / 进行中 / 已交付 / 已闭环 / 未闭环逾期 / 请假顺延
闭环五要素	多选	勾齐五项才允许置为「已闭环」
mentor 打分	数字	1-5，权威分
agent 建议分	数字	1-5，仅参考，不覆盖 mentor
mentor 评语	文本	回写 wiki 的原文
外部成果	文本	大目标专用：用户数 / 金额
友商对比	文本	大目标专用
任务 GUID	文本	对应的飞书任务，交付状态回查用

5.2 一个 mentor 一张独立台账 (隔离靠文件边界，不靠视图)

不做「一张大表 + 筛选视图」。飞书的行级隔离要靠多维表格的自定义角色，而自定义角色必须先开高级权限，且 base 一旦放进 wiki 或被嵌进文档就开不了（错误码 1254301）。按 mentor 拆成独立 base，隔离退化成「文件给谁看」这件飞书原生就保证的事，不依赖任何高级特性。

要素	做法
命名	「TODO 台账-<mentor 姓名>」，一个 mentor 一个 base，表内一张 todo 表
开通	feishu_mentor_ledger_ensure 幂等开通：已存在则直接返回 app_token，不会建出第二张
建表方式	维护一份列已建好的模板 base，用 POST /apps/:app_token/copy (without_content=true) 复制，而不是每次重建列
授权	只给三方：该 mentor（编辑）、boss（只读）、机器人（编辑）。其他人不加，等于默认不可见
投递	agent 按 mentor_open_id 私聊推送，链接与卡片都只发给本人；不进群、不群发
boss 视角	boss 不写 todo、不收派发；单独一张全公司汇总 base（只读）+ 一张整体统计卡

- 统计卡内容：本周大目标/小目标/todo 各多少条、已闭环多少、逾期多少、请假顺延多少、下属平均分。

- 统计图用 feishu_chart 渲染 PNG（进度条 / 分组柱），附在卡片里，趋势一眼可见。
- mentor 换人或下属调整时重跑 ensure：授权按新名单增删，历史数据留在原 base 不迁移。

代价说明：base 数量等于 mentor 数量，跨 mentor 的全公司统计需要 agent 逐个 base 读取后合并，而不是一次查询。这是为「互不可见」付的确定代价，换来不依赖高级权限。

6. 派发与交付设计（R5 / R6）

每个负责人收到一张 TODO 卡：一张卡列出他本周期全部 todo，每条一个勾选框、一个详情链接、一个对应的飞书原生任务。这正是 feishu_todo_card_send 的能力（已在 main 上）：勾一条只结那一条，其余仍可勾，卡片原地更新。

6.1 派发顺序（顺序不能颠倒）

- 1. 先建飞书任务：对每条 todo 调 feishu_task_create，拿回 task_guid，并把截止时间写进任务。
- 2. 再发 TODO 卡：feishu_todo_card_send 把 task_guid 填进对应行，行标题即链到该任务。
- 3. 回写台账：把 task_guid 存进 Bitable 的「任务 GUID」列，供后续查交付状态。
- 4. 建截止提醒：对每条 todo 用 schedule_manage 建 once_at + fire=tool 的一次性提醒，截止前一天私聊提醒本人。

顺序理由：卡片行链接指向任务，任务不存在就没有可点的目标；task/v2 的返回体里没有 web URL，行链接用官方 applink 协议拼（这一点在 feishu-todo-card 技能里已注明未经真实任务实测）。

6.2 请假的人怎么标（R5 的硬要求）

请假必须在派发前判定，否则会给休假的人派活并在报表里记逾期。判定后分三种处理：

情形	卡片与任务	台账状态	报表呈现
整周期请假	不派卡、不建任务	请假顺延	该人整块标「请假」，不计入逾期
部分日期请假	照常派，截止日顺延请假天数	请假顺延	该条标「截止顺延 N 天」
请假中已交付	照常，不特殊处理	按实际状态	不标请假

请假事实来自 todo 表格里新增的「请假表」子表，由 feishu_leave_query 读取并判定命中日期——已定案，详见 8.1。

6.3 交付与评价回收

- 执行人勾选卡片行 → 框架把该行改成已完成态并原地更新卡片 → 派发 feishu_todo_card_tick → 标记飞书任务完成。

- 连点会被合并成一个批 (<feishu_card_action_batch>)：每条都要逐个处理，但只回一条消息——这是框架已有约定。
- 交付后给 mentor 发一张评价卡：按钮为 1-5 分，另有「打回重做」；评语走卡片表单一次提交。
- mentor 提交后 agent 做两件事：写 Bitable 的打分与评语字段；把「评分 + 评语」追加到该人 wiki 快照页对应 todo 之后。
- agent 建议分只作参考：按「是否按期、是否一次通过、成果物是否齐备」给 1-5，与 mentor 分并列存放，冲突时以 mentor 为准。

7. 闭环判定与回流 (R7)



图 3 公司 TODO 管理闭环流程图

7.1 闭环五要素

「交付完成」不等于「勾了框」。五项齐备才置为已闭环，缺任一项都停在未闭环：

#	要素	从哪验证
1	有明确验收人	台账 mentor 字段非空
2	有自设截止时间且已到期或提前完成	台账截止日期 + 实际完成时间
3	负责人已勾选并提交成果物	飞书任务 completed_at 非空
4	mentor 已给出评价与打分	台账打分与评语字段非空
5	评价已回写进本人 wiki 该条 todo 之后	wiki_read 该快照页命中评语文本

7.2 未闭环的回流规则

- 下一周期开始前跑一次审计：遍历上一周期所有未闭环项，按原截止日算出逾期天数。
- 回流项进入新周期的派发列表，标题前缀「[逾期 N 天]」，原截止日保留在正文里，不许静默改截止日。
- 连续两个周期未闭环的项，在 mentor 报表里单独列一块「持续逾期」，并同步给该 mentor 的上级。
- 请假顺延项不计入逾期天数，但仍然回流——顺延不等于取消。
- 回流不删除历史：上一周期快照页里该条仍是未闭环，新周期页里是新的一条，两条通过标题可对应。

8. 能力边界与已定方案

以下三条是我在 main 上逐个核实过的事实，不是猜测。三条都已定案，但都会影响日常口径，需要知情。

8.1 请假来源：表格加「请假表」+ 专用工具（已定）

先说事实：飞书开放平台的 /approval/v4/instances 只有「建实例」（POST）和「按 instance_id 单查」，没有按人枚举的接口；feishu_attendance_query 返回的是打卡结果（正常/迟到/早退/缺卡），不是请假单。所以「从系统里自动查出谁请假」这条路在当前接口下走不通，不是工具没做，是接口没有。

定案做法：把请假变成和 todo 同源的一条事实。

环节	做法
载体	在那张 todo 表格里加一张「请假表」子表，与 todo 表同一个 sheet_token
字段	姓名 / 开始日期 / 结束日期 / 类型（事假·病假·年假·出差） / 是否整天 / 备注
填写	员工本人填，mentor 可代填；填写时机与周一三五填 todo 相同，不新增一次动作
读取	feishu_leave_query 读该子表，按 [date_from, date_to] 求交集，返回每人命中的具体日期
使用点	派发前（6.2 三种情形）、报表口径（请假顺延不计逾期）、闭环判定（7.2 命中日不计入逾期天数）
留白	依赖人工填写；漏填等于视为未请假。可选加固：审计技能对「整周期零交付且无请假记录」的人单独提示 mentor 复核

为什么不用打卡反推做兜底：缺卡同时对应出差、外勤、忘打卡和请假，反推会把前三种误判成请假并静默顺延截止日，错误方向恰好是「放宽考核」，比漏判更难被发现。宁可漏填时报逾期，由本人当场申诉。

8.2 wiki 无 append（已解决，但需知情）

wiki_write 只能整页覆盖。方案用「每次一页快照 + 汇总页只改索引区块」绕开，代价是页数随周期线性增长（100 人 × 每周 3 次 ≈ 每年 1.5 万页），以及同一人的汇总页不能并发改写。规模到万级页后 wiki_search 的响应会变慢，届时可按年份归档（把往年快照页移出主索引）。

8.3 mentor 之间的隔离：拆独立 base（已定）

同一张 base 内做行级隔离要用自定义角色，而自定义角色的前置是开高级权限，且 base 放进 wiki 或被嵌进文档时开不了（错误码 1254301）。筛选视图不是权限——拿到链接的人能看到整表。因此按 mentor 拆独立 base（做法见 5.2）：隔离由飞书原生的文件权限保证，不依赖高级权限，也不存在「链接泄漏即全表泄漏」。

需要知情的两点：一是 base 数量随 mentor 数线性增长，全公司统计要逐个 base 读取再合并，不是一次查询；二是 mentor 换人或下属调整时要重跑 feishu_mentor_ledger_ensure 调授权，历史数据留在原 base 不迁移。

9. 落地实施步骤

全部改动落在 agent 包内（tools / skills / schedules），框架代码一行不改：

步骤	产出	所用工具	验收方式
1	todo 表格加「请假表」子表与六个字段	人工	feishu_sheet_read 能读到该子表
2	新增工具 feishu_leave_query	写入 tools/ 目录	构造跨周期请假，命中日期算对
3	建一份台账模板 base（列建好、无数数据）	feishu_bitable_create_table / _update_field	字段齐备，能手工写入一行
4	新增工具 feishu_mentor_ledger_ensure	写入 tools/ 目录	连跑两次只出一张台账，授权恰好三方
5	建组织根页与人员页骨架	wiki_write	wiki_links 能从根页走到每个人
6	写 SKILL company-todo-sync	skill_manage	手工触发一次，采集→建树→报表→派发全过
7	写 SKILL company-todo-review	skill_manage	模拟一次 mentor 打分并回写 wiki
8	写 SKILL company-todo-audit	skill_manage	构造一条未闭环项，确认回流
9	建三条 schedule	schedule_manage	cron 0 15 * * 1,3,5 到点触发
10	小范围试运行一个周期	—	一个团队跑通，再全公司铺开

强烈建议第 8 步不要省：先用一个团队（一个 mentor + 3-5 人）跑满一个周期，把表格填写口径、请假标注、评分习惯这三件最容易返工的事验证掉，再全公司推。

9.1 三条 schedule 的具体配置

名称	cron / 触发	fire	做什么
todo-cycle-sync	0 15 * * 1,3,5	prompt	采集→建树→报表→派发（第 4/3/5/6 节）
todo-cycle-audit	30 14 * * 1,3,5	prompt	先于采集半小时跑闭环判定与回流
todo-deadline-remind	once_at 逐条建	tool	截止前一天私聊提醒本人

注意 audit 排在 sync 之前：回流项必须先算出来，才能进入本次派发列表。另外 once_at 的一次性任务必须同时给 fire=tool（框架不接受 prompt 型一次性任务）。

10. 主要风险与应对

风险	后果	应对
表格填写口径不一致	解析层抽不出截止日/验收人	试运行阶段固化模板；缺字段标 needs_fix 不静默丢
mentor 不按时打分	闭环第 4 项永远缺，全部挂未闭环	评价卡超时未回则升级提醒其上级
请假误判	给休假者派活并记逾期	选方案 A 由本人填报，不用打卡反推
同一人汇总页并发改写	后写覆盖先写，丢一条索引	编排层按人串行；写前 wiki_read 校验
wiki 页数膨胀	搜索变慢	按年归档往年快照页
卡片 applink 未实测	行内详情链接可能点不开	试运行时用真实 task_guid 验证一次

文档结束。本方案所有工具名与能力边界均在 psi-agent origin/main (fd6ef94f) 上核实；PR 623 的 TODO 卡能力已随 bd1ba045 合入 main，故第 6 节不再依赖未合分支。